AI Assisted Coding

Lab Assignment 12.5

Name : Vineeth Chidurala **Date : 13/03/2026**

Hall Ticket no : 2303A52447

Task Description #1 (Sorting – Merge Sort Implementation)

• Task: Use AI to generate a Python program that implements the Merge Sort algorithm.

• Instructions:

o Prompt AI to create a function merge_sort(arr) that sorts a list in ascending order.

o Ask AI to include time complexity and space complexity in the function docstring.

o Verify the generated code with test cases.

• Expected Output:

o A functional Python script implementing Merge Sort with proper documentation.

Prompt :generate a Python program that implements the Merge Sort algorithm. and verify using test cases

```python
# generate a Python program that implements the Merge Sort algorithm. and verify using test cases
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid])
    right_half = merge_sort(arr[mid:])

    return merge(left_half, right_half)
def merge(left, right):
    merged = []
    left_index = right_index = 0

    while left_index < len(left) and right_index < len(right):
        if left[left_index] < right[right_index]:
            merged.append(left[left_index])
            left_index += 1
        else:
            merged.append(right[right_index])
            right_index += 1

    merged.extend(left[left_index:])
    merged.extend(right[right_index:])

    return merged
```

Output :

```
✓ 0.0s

Test case 1: [38, 27, 43, 3, 9, 82, 10] -> [3, 9, 10, 27, 38, 43, 82]
Test case 2: [5, 2, 9, 1, 5, 6] -> [1, 2, 5, 5, 6, 9]
Test case 3: [12, 11, 13, 5, 6, 7] -> [5, 6, 7, 11, 12, 13]
Test case 4: [] -> []
Test case 5: [1] -> [1]
Test case 6: [2, 1] -> [1, 2]
```

Task #2 (Searching – Binary Search with AI Optimization)

• Task: Use AI to create a binary search function that finds a target element in a sorted list.

• Instructions:Prompt AI to create a function binary_search(arr, target) returning the index of the target or -1 if not found.Include docstrings explaining best, average, and worst-case complexities. Test with various inputs.

• Expected Output:

Python code implementing binary search with AI-

generated comments and docstrings.

Prompt :

generate a binary search function that finds a target element in a sorted list.

```python
def binary_search(arr, target):
    left, right = 0, len(arr) - 1

    while left <= right:
        mid = left + (right - left) // 2

        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return -1  # Target not found
# Test cases to verify the implementation
if __name__ == "__main__":
    sorted_list = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
    targets = [5, 1, 10, 11]

    for target in targets:
        result = binary_search(sorted_list, target)
        if result != -1:
            print(f"Target {target} found at index: {result}")
        else:
            print(f"Target {target} not found in the list.")
```

Output :

```
✓ 0.0s

Target 5 found at index: 4
Target 1 found at index: 0
Target 10 found at index: 9
Target 11 not found in the list.
```

Task 3: Smart Healthcare Appointment Scheduling System

A healthcare platform maintains appointment records containing appointment ID, patient name, doctor name, appointment time, and consultation fee. The system needs to:

1. Search appointments using appointment ID.
2. Sort appointments based on time or consultation fee.

Student Task

• Use AI to recommend suitable searching and sorting algorithms.

• Justify the selected algorithms.

• Implement the algorithms in Python

Prompt :

A healthcare platform maintains appointment records containing appointment ID, patient name, doctor name, appointment time, and consultation fee. The system needs to: 1. Search appointments using appointment ID. 2. Sort appointments based on time or consultation fee.

```python
class Appointment:
    def __init__(self, appointment_id, patient_name, doctor_name, appointment_time, consultation_fee):
        self.appointment_id = appointment_id
        self.patient_name = patient_name
        self.doctor_name = doctor_name
        self.appointment_time = appointment_time
        self.consultation_fee = consultation_fee
class AppointmentManager:
    def __init__(self):
        self.appointments = []
    def add_appointment(self, appointment):
        self.appointments.append(appointment)
    def search_appointment_by_id(self, appointment_id):
        for appointment in self.appointments:
            if appointment.appointment_id == appointment_id:
                return appointment
        return None  # Appointment not found
    def sort_appointments_by_time(self):
        self.appointments.sort(key=lambda x: x.appointment_time)
    def sort_appointments_by_fee(self):
        self.appointments.sort(key=lambda x: x.consultation_fee)
# Test cases to verify the implementation
if __name__ == "__main__":
    appointment_manager = AppointmentManager()

    # Adding some appointments
    appointment_manager.add_appointment(Appointment(1, "John Doe", "Dr. Smith", "2023-10-01 10:00", 100))
    appointment_manager.add_appointment(Appointment(2, "Jane Doe", "Dr. Brown", "2023-10-01 11:00", 150))
    appointment_manager.add_appointment(Appointment(3, "Alice Smith", "Dr. Johnson", "2023-10-01 09:00", 120))

    # Searching for an appointment by ID
    search_result = appointment_manager.search_appointment_by_id(2)
    if search_result:
        print(f"Appointment found: {search_result.patient_name} with {search_result.doctor_name} at {search_result.appointment_time} for $
    else:
        print("Appointment not found.")
```

Output :

```
✓ 0.0s
Appointment found: Jane Doe with Dr. Brown at 2023-10-01 11:00 for $150
Appointments sorted by time:
Alice Smith with Dr. Johnson at 2023-10-01 09:00 for $120
John Doe with Dr. Smith at 2023-10-01 10:00 for $100
Jane Doe with Dr. Brown at 2023-10-01 11:00 for $150
Appointments sorted by consultation fee:
John Doe with Dr. Smith at 2023-10-01 10:00 for $100
Alice Smith with Dr. Johnson at 2023-10-01 09:00 for $120
Jane Doe with Dr. Brown at 2023-10-01 11:00 for $150
```

Task 4: Railway Ticket Reservation System

Scenario

A railway reservation system stores booking details such as ticket ID, passenger name, train number, seat number, and travel date. The system must:

1. Search tickets using ticket ID.

2. Sort bookings based on travel date or seat number.

Student Task

• Identify efficient algorithms using AI assistance.

• Justify the algorithm choices.

• Implement searching and sorting in Python.

Prompt :

A railway reservation system stores booking details such as ticket ID, passenger name, train number, seat number, and travel date. The system must: 1.Search tickets using ticket ID. 2. Sort bookings based on travel date or seat number implement a Python program to achieve this functionality.search and sorting functionality for a railway reservation system.

```python
class Booking:
    def __init__(self, ticket_id, passenger_name, train_number, seat_number, travel_date):
        self.ticket_id = ticket_id
        self.passenger_name = passenger_name
        self.train_number = train_number
        self.seat_number = seat_number
        self.travel_date = travel_date
class BookingManager:
    def __init__(self):
        self.bookings = []
    def add_booking(self, booking):
        self.bookings.append(booking)
    def search_booking_by_ticket_id(self, ticket_id):
        for booking in self.bookings:
            if booking.ticket_id == ticket_id:
                return booking
        return None  # Booking not found
    def sort_bookings_by_travel_date(self):
        self.bookings.sort(key=lambda x: x.travel_date)
    def sort_bookings_by_seat_number(self):
        self.bookings.sort(key=lambda x: x.seat_number)
# Test cases to verify the implementation
if __name__ == "__main__":
    booking_manager = BookingManager()

    # Adding some bookings
    booking_manager.add_booking(Booking(1, "John Doe", "Train A", "12A", "2023-10-01"))
    booking_manager.add_booking(Booking(2, "Jane Doe", "Train B", "15B", "2023-10-02"))
    booking_manager.add_booking(Booking(3, "Alice Smith", "Train A", "10C", "2023-10-01"))

    # Searching for a booking by ticket ID
    search_result = booking_manager.search_booking_by_ticket_id(2)
    if search_result:
        print(f"Booking found: {search_result.passenger_name} on {search_result.train_number} seat {search_result.seat_number} for travel
    else:
        print("Booking not found.")
```

Output :

```
✓ 0.0s

Booking found: Jane Doe on Train B seat 15B for travel date 2023-10-02
Bookings sorted by travel date:
John Doe on Train A seat 12A for travel date 2023-10-01
Alice Smith on Train A seat 10C for travel date 2023-10-01
Jane Doe on Train B seat 15B for travel date 2023-10-02
Bookings sorted by seat number:
Alice Smith on Train A seat 10C for travel date 2023-10-01
John Doe on Train A seat 12A for travel date 2023-10-01
Jane Doe on Train B seat 15B for travel date 2023-10-02
```

Task 5: Smart Hostel Room Allocation System

A hostel management system stores student room allocation details including student ID, room number, floor, and allocation date. The system needs to:

1. Search allocation details using student ID.
2. Sort records based on room number or allocation date.

Student Task

• Use AI to suggest optimized algorithms.

• Justify the selections.

• Implement the solution in Python

Prompt :

A hostel management system stores student room allocation details including student ID, room number, floor, and allocation date. The system needs to:1. Search allocation details using student ID.2. Sort records based on room number or allocation date

```python
class RoomAllocation:
    def __init__(self, student_id, room_number, floor, allocation_date):
        self.student_id = student_id
        self.room_number = room_number
        self.floor = floor
        self.allocation_date = allocation_date
class HostelManagementSystem:
    def __init__(self):
        self.allocations = []
    def add_allocation(self, allocation):
        self.allocations.append(allocation)
    def search_allocation_by_student_id(self, student_id):
        for allocation in self.allocations:
            if allocation.student_id == student_id:
                return allocation
        return None  # Allocation not found
    def sort_allocations_by_room_number(self):
        self.allocations.sort(key=lambda x: x.room_number)
    def sort_allocations_by_allocation_date(self):
        self.allocations.sort(key=lambda x: x.allocation_date)
# Test cases to verify the implementation
if __name__ == "__main__":
    hostel_system = HostelManagementSystem()

    # Adding some allocations
    hostel_system.add_allocation(RoomAllocation(1, "101", "1st Floor", "2023-10-01"))
    hostel_system.add_allocation(RoomAllocation(2, "102", "1st Floor", "2023-10-02"))
    hostel_system.add_allocation(RoomAllocation(3, "201", "2nd Floor", "2023-10-01"))

    # Searching for an allocation by student ID
    search_result = hostel_system.search_allocation_by_student_id(2)
    if search_result:
        print(f"Allocation found: Student ID {search_result.student_id} in room {search_result.room_number} on {search_result.floor} alloc
    else:
        print("Allocation not found.")
```

Output :

```
✓ 0.0s

Allocation found: Student ID 2 in room 102 on 1st Floor allocated on 2023-10-02
Allocations sorted by room number:
Student ID 1 in room 101 on 1st Floor allocated on 2023-10-01
Student ID 2 in room 102 on 1st Floor allocated on 2023-10-02
Student ID 3 in room 201 on 2nd Floor allocated on 2023-10-01
Allocations sorted by allocation date:
Student ID 1 in room 101 on 1st Floor allocated on 2023-10-01
Student ID 3 in room 201 on 2nd Floor allocated on 2023-10-01
Student ID 2 in room 102 on 1st Floor allocated on 2023-10-02
```

Task 6: Online Movie Streaming Platform

A streaming service maintains movie records with movie ID, title, genre, rating, and release year. The platform needs to:
1. Search movies by movie ID.
2. Sort movies based on rating or release year.

Student Task
- Recommend searching and sorting algorithms using AI.
- Justify the chosen algorithms.
- Implement Python functions

Prompt :

A streaming service maintains movie records with movie ID, title, genre, rating, and release year. The platform needs to: 1. Search movies by movie ID. 2. Sort movies based on rating or release year.analyze the above senario and requirements and Implement Python functions.

```python
class Movie:
    def __init__(self, movie_id, title, genre, rating, release_year):
        self.movie_id = movie_id
        self.title = title
        self.genre = genre
        self.rating = rating
        self.release_year = release_year
class StreamingService:
    def __init__(self):
        self.movies = []
    def add_movie(self, movie):
        self.movies.append(movie)
    def search_movie_by_id(self, movie_id):
        for movie in self.movies:
            if movie.movie_id == movie_id:
                return movie
        return None  # Movie not found
    def sort_movies_by_rating(self):
        self.movies.sort(key=lambda x: x.rating, reverse=True)
    def sort_movies_by_release_year(self):
        self.movies.sort(key=lambda x: x.release_year)
# Test cases to verify the implementation
if __name__ == "__main__":
    streaming_service = StreamingService()

    # Adding some movies
    streaming_service.add_movie(Movie(1, "Movie A", "Action", 8.5, 2020))
    streaming_service.add_movie(Movie(2, "Movie B", "Comedy", 7.2, 2019))
    streaming_service.add_movie(Movie(3, "Movie C", "Drama", 9.0, 2021))
```

Output :

```
✓ 0.0s
Movie found: Movie B (Comedy) with rating 7.2 released in 2019
Movies sorted by rating:
Movie C (Drama) with rating 9.0 released in 2021
Movie A (Action) with rating 8.5 released in 2020
Movie B (Comedy) with rating 7.2 released in 2019
Movies sorted by release year:
Movie B (Comedy) with rating 7.2 released in 2019
Movie A (Action) with rating 8.5 released in 2020
Movie C (Drama) with rating 9.0 released in 2021
```

Task 7: Smart Agriculture Crop Monitoring System

An agriculture monitoring system stores crop data with crop ID, crop name, soil moisture level, temperature, and yield estimate. Farmers need to:

1. Search crop details using crop ID.
2. Sort crops based on moisture level or yield estimate.

Student Task

- Use AI-assisted reasoning to select algorithms.
- Justify algorithm suitability.
- Implement searching and sorting in Python.

Prompt :

An agriculture monitoring system stores crop data with crop ID, crop name, soil moisture level, temperature, and yield estimate. Farmers need to:1. Search crop details using crop ID.2. Sort crops based on moisture level or yield estimate.analyze the above senario and requirements and Implement searching and sorting in Python.

```
(parameter) crop_name: Any
class Crop:
    def __init__(self, crop_id, crop_name, soil_moisture_level, temperature, yield_estimate):
        self.crop_id = crop_id
        self.crop_name = crop_name
        self.soil_moisture_level = soil_moisture_level
        self.temperature = temperature
        self.yield_estimate = yield_estimate
class AgricultureMonitoringSystem:
    def __init__(self):
        self.crops = []
    def add_crop(self, crop):
        self.crops.append(crop)
    def search_crop_by_id(self, crop_id):
        for crop in self.crops:
            if crop.crop_id == crop_id:
                return crop
        return None  # Crop not found
    def sort_crops_by_moisture_level(self):
        self.crops.sort(key=lambda x: x.soil_moisture_level)
    def sort_crops_by_yield_estimate(self):
        self.crops.sort(key=lambda x: x.yield_estimate, reverse=True)
# Test cases to verify the implementation
if __name__ == "__main__":
    monitoring_system = AgricultureMonitoringSystem()
```

Output :

```
✓ 0.0s

Crop found: Corn with soil moisture level 40, temperature 28, and yield estimate 1500
Crops sorted by soil moisture level:
Wheat with soil moisture level 30, temperature 25, and yield estimate 1000
Rice with soil moisture level 35, temperature 30, and yield estimate 1200
Corn with soil moisture level 40, temperature 28, and yield estimate 1500
Crops sorted by yield estimate:
Corn with soil moisture level 40, temperature 28, and yield estimate 1500
Rice with soil moisture level 35, temperature 30, and yield estimate 1200
Wheat with soil moisture level 30, temperature 25, and yield estimate 1000
```

Task 8: Airport Flight Management System

An airport system stores flight information including flight ID, airline name, departure time, arrival time, and status. The system must:

1. Search flight details using flight ID.
2. Sort flights based on departure time or arrival time.

Student Task

- Use AI to recommend algorithms.
- Justify the algorithm selection.
- Implement searching and sorting logic in Python.

Prompt :

An airport system stores flight information including flight ID,airline name, departure time, arrival time, and status. The system must:1. Search flight details using flight ID.2. Sort flights based on departure time or arrival timeanalyze the above senario and Implement searching and sorting logic in Python

```python
class Flight:
    def __init__(self, flight_id, airline_name, departure_time, arrival_time, status):
        self.flight_id = flight_id
        self.airline_name = airline_name
        self.departure_time = departure_time
        self.arrival_time = arrival_time
        self.status = status
class AirportSystem:
    def __init__(self):
        self.flights = []
    def add_flight(self, flight):
        self.flights.append(flight)
    def search_flight_by_id(self, flight_id):
        for flight in self.flights:
            if flight.flight_id == flight_id:
                return flight
        return None  # Flight not found
    def sort_flights_by_departure_time(self):
        self.flights.sort(key=lambda x: x.departure_time)
    def sort_flights_by_arrival_time(self):
        self.flights.sort(key=lambda x: x.arrival_time)
# Test cases to verify the implementation
```

```python
# Test cases to verify the implementation
if __name__ == "__main__":
    airport_system = AirportSystem()

    # Adding some flights
    airport_system.add_flight(Flight(1, "Airline A", "2023-10-01 10:00", "2023-10-01 12:00", "On Time"))
    airport_system.add_flight(Flight(2, "Airline B", "2023-10-01 11:00", "2023-10-01 13:00", "Delayed"))
    airport_system.add_flight(Flight(3, "Airline C", "2023-10-01 09:00", "2023-10-01 11:00", "On Time"))

    # Searching for a flight by ID
    search_result = airport_system.search_flight_by_id(2)
    if search_result:
        print(f"Flight found: {search_result.airline_name} departing at {search_result.departure_time} arriving at {search_result.arrival_
    else:
        print("Flight not found.")

    # Sorting flights by departure time
    airport_system.sort_flights_by_departure_time()
    print("Flights sorted by departure time:")
    for flight in airport_system.flights:
        print(f"{flight.airline_name} departing at {flight.departure_time} arriving at {flight.arrival_time} with status {flight.status}")

    # Sorting flights by arrival time
    airport_system.sort_flights_by_arrival_time()
    print("Flights sorted by arrival time:")
    for flight in airport_system.flights:
        print(f"{flight.airline_name} departing at {flight.departure_time} arriving at {flight.arrival_time} with status {flight.status}")
```

Output :

```
✓ 0.0s

Flight found: Airline B departing at 2023-10-01 11:00 arriving at 2023-10-01 13:00 with status Delayed
Flights sorted by departure time:
Airline C departing at 2023-10-01 09:00 arriving at 2023-10-01 11:00 with status On Time
Airline A departing at 2023-10-01 10:00 arriving at 2023-10-01 12:00 with status On Time
Airline B departing at 2023-10-01 11:00 arriving at 2023-10-01 13:00 with status Delayed
Flights sorted by arrival time:
Airline C departing at 2023-10-01 09:00 arriving at 2023-10-01 11:00 with status On Time
Airline A departing at 2023-10-01 10:00 arriving at 2023-10-01 12:00 with status On Time
Airline B departing at 2023-10-01 11:00 arriving at 2023-10-01 13:00 with status Delayed
```